CAMPUS RECRUITMENT TRAINING & PLACEMENT BOARD

OFFICIAL PRESCRIBED PLACEMENT MASTER TEXTBOOK

Modern JavaScript & TypeScript: Full-Stack Placement Edition (Prescribed Textbook)

Subject Track: JavaScript / TypeScript

Edition: 2026 Campus Placement Edition

SYLLABUS & CURRICULUM OVERVIEW

Complete textbook on JavaScript runtime engine architecture, V8 microtask queue, closures, prototypal inheritance, ES6+, TypeScript static typing, DOM event loop, and polyfills.

STRICT EXAMINATION GROUNDING GUARANTEE

All campus placement assessments, live code challenges, milestone tests, and company interview drills for JavaScript / TypeScript on this platform are derived strictly and exclusively from the theoretical concepts, algorithms, worked examples, and placement problems documented in this official textbook. Mastery of these chapters guarantees 100% exam preparation.

PRESCRIBED CHAPTER SYLLABUS (8 CHAPTERS):

Ch 1: JavaScript Engine Architecture, Execution Context & Event Loop

Ch 2: Scopes, Closures & Prototype Inheritance

Ch 3: Asynchronous JavaScript: Promises & async/await

Ch 4: Modern ES6+ Features & Functional Programming

Ch 5: TypeScript Type System & Strict Safety

Ch 6: DOM Manipulation, Browser APIs & Event Propagation

Ch 7: Modules, Bundlers & Memory Management

Ch 8: Solved Frontend Placement Coding Drills & System Design

Author: Prescribed Courseware Architectural Board

Verified Complete Unabridged Placement Master Edition | Internal Campus Circulation

Edition: 2026 Campus Placement Edition

Subject: JavaScript / TypeScript

Courseware Board: Prescribed Placement Architectural Board

Status: In-App Complete Master Textbook (Full Curriculum Available In-Platform)

> **? EXAM GROUNDING POLICY:** All campus placement technical assessments, coding lab challenges, and technical interview evaluations are strictly grounded in the concepts, algorithms, formulas, step-by-step solved examples, practice questions, and interview questions detailed in this textbook.

· Course Overview & Curriculum Scope

Complete textbook on JavaScript runtime engine architecture, V8 microtask queue, closures, prototypal inheritance, ES6+, TypeScript static typing, DOM event loop, and polyfills.

· Complete Table of Contents (Chapter-by-Chapter)

- [Chapter 1: JavaScript Engine Architecture, Execution Context & Event Loop](#chapter-1)
- [Chapter 2: Scopes, Closures & Prototype Inheritance](#chapter-2)
- [Chapter 3: Asynchronous JavaScript: Promises & async/await](#chapter-3)
- [Chapter 4: Modern ES6+ Features & Functional Programming](#chapter-4)
- [Chapter 5: TypeScript Type System & Strict Safety](#chapter-5)
- [Chapter 6: DOM Manipulation, Browser APIs & Event Propagation](#chapter-6)
- [Chapter 7: Modules, Bundlers & Memory Management](#chapter-7)
- [Chapter 8: Solved Frontend Placement Coding Drills & System Design](#chapter-8)

CHAPTER 1

JavaScript Engine Architecture, Execution Context & Event Loop

Key Principle / Exam Takeaway: Master V8 Call Stack, Memory Heap, Microtask Queue (Promises) vs Macrotask Queue (setTimeout), and execution contexts.

Core Curriculum Topics Covered in this Chapter:

- V8 Engine architecture: Ignition interpreter, TurboFan optimizing compiler
- Execution Context: Creation phase (hoisting), Execution phase, Call Stack
- The Event Loop: Single-threaded non-blocking runtime dynamics
- Task Queues: Microtask Queue (process.nextTick, Promise.then) vs Macrotask Queue (setTimeout, setInterval, I/O)
- Hoisting: var (undefined) vs let/const (Temporal Dead Zone - TDZ)

· 1. Core Concepts & In-Depth Explanation

JavaScript is a single-threaded, non-blocking, asynchronous concurrent runtime powered by an engine such as Google's V8.

Execution Context & Hoisting: When JavaScript runs code, it creates an Execution Context in two phases: 1. **Creation Phase:** Memory is allocated for variables and functions. Function declarations are hoisted completely into memory. Variables declared with `var` are initialized to `undefined`. Variables declared with `let` and `const` are hoisted but uninitialized, placing them in the **Temporal Dead Zone (TDZ)** where accessing them before declaration throws `ReferenceError`. 2. **Execution Phase:** Code is evaluated line-by-line.

The Event Loop: Because the Call Stack is single-threaded, asynchronous callbacks are offloaded to Web APIs. Upon completion, callbacks enter queues:

- **Microtask Queue:** Promise callbacks (`.then()`, `.catch()`, `async/await`), `queueMicrotask`, `MutationObserver`.
- **Macrotask Queue:** `setTimeout`, `setInterval`, `setImmediate`, DOM events.

The Event Loop continuously checks: whenever the Call Stack empties, it drains the **entire Microtask Queue** before picking a single task from the Macrotask Queue!

· 2. Key Formulas, Rules & Complexity Classes

1. **Event Loop Priority Rule:** $$\text{Call Stack Empty} \implies \text{Flush ALL Microtasks} \to \text{Render Frame (if due)} \to \text{Execute ONE Macrotask}$$ 2. **Temporal Dead Zone (TDZ):** $$\text{Start of block} \to \text{Variable Declaration} \implies \text{TDZ (Access throws ReferenceError)}$$

· 3. Step-by-Step Solved Examples

Example 1.1: Predicting Event Loop Output with Promises and Timers

- **Difficulty / Target Companies:** `Medium | Amazon, Google, Uber`
- **Problem Statement:** Determine the exact console output order of the following snippet.

Step-by-Step Walkthrough & Logic:

- Step 1: `console.log(1)` is synchronous; logs 1 immediately.
- Step 2: `setTimeout` queues a Macrotask callback for logging 2.
- Step 3: `Promise.resolve().then(...)` queues a Microtask callback for logging 3.
- Step 4: `console.log(4)` is synchronous; logs 4 immediately.
- Step 5: Call stack empty. Drains Microtask Queue: logs 3.
- Step 6: Microtasks clear. Picks Macrotask: logs 2.

Complete Solved Code / Calculation:

```
console.log(1);

setTimeout(() => {
    console.log(2);
}, 0);

Promise.resolve().then(() => {
    console.log(3);
});

console.log(4);

// Output:
// 1
// 4
// 3
// 2
```

- **Line-by-Line Breakdown & Complexity Analysis:** Synchronous code runs first (1, 4), then Microtasks execute before Macrotasks (3, then 2).

· 4. Practice Questions with Answers & Explanations

Practice Question 1.1

Question: What happens if a microtask continually schedules another microtask inside a recursive loop?

Options:

- A) Browser automatically stops after 100 iterations
- B) Starves the event loop, freezing UI rendering and preventing macrotasks from running
- C) Throws Maximum Call Stack Exceeded error
- D) Offloads to Web Worker

Answer: **B) Starves the event loop, freezing UI rendering and preventing macrotasks from running** **Detailed Solution & Reasoning:** The event loop must exhaust all microtasks before yielding control, so an endless microtask loop starves macrotasks and DOM repaints.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 1.1 (Amazon)

- **Interview Question:** _Explain the Temporal Dead Zone (TDZ) and why let/const differ from var._

- **Model High-Scoring Answer:** Variables declared with `let` and `const` are hoisted to the top of their block scope during the creation phase, but unlike `var` (which is initialized to `undefined`), they remain completely uninitialized. The TDZ is the span of code from the start of the enclosing block until the line where the variable is explicitly declared. Accessing the variable within the TDZ throws a `ReferenceError`.

- **Interviewer Follow-Up & Pro Tip:** ? Cite block scoping and preventing use before initialization.

CHAPTER 2

Scopes, Closures & Prototype Inheritance

Key Principle / Exam Takeaway: Master lexical scope, closures, prototype chain (__proto__ vs prototype), Object.create, and Function.prototype.bind/call/apply.

Core Curriculum Topics Covered in this Chapter:

- Lexical Scoping and Scope Chain traversal
- Closures: Private variables and module patterns
- Prototype Chain: Object.prototype, Function.prototype, constructor functions
- Inheritance in ES5 (Object.create, call) vs ES6 class syntax (syntactic sugar)
- this binding rules: Default, Implicit, Explicit (call/apply/bind), and Arrow functions

· 1. Core Concepts & In-Depth Explanation

JavaScript uses **Lexical Scoping**: variable access is determined by physical code structure at write time. A **Closure** is created when an inner function retains access to variables from its outer lexical scope even after the outer function has finished executing and returned.

Prototype Inheritance: Unlike class-based languages (C++, Java), JavaScript uses prototypical inheritance: objects inherit properties directly from other objects via an internal link (`[[Prototype]]`, accessible via `Object.getPrototypeOf(obj)` or `__proto__`). When a property is queried on an object, JavaScript checks the object itself; if missing, it walks up the prototype chain until reaching `Object.prototype` (whose prototype is `null`).

The `this` Keyword: `this` is determined dynamically at call time based on call-site: 1. `obj.method()`: Implicit binding to `obj`. 2. `fn.call(ctx, ...)`: Explicit binding to `ctx`. 3. `new Constructor()`: Bound to newly allocated object. 4. Arrow functions: Do NOT have their own `this`; they inherit `this` lexically from enclosing scope.

· 2. Key Formulas, Rules & Complexity Classes

1. **Prototype Chain Resolution:** $$\text{obj.prop} \implies \text{obj} \to \text{obj.__proto__} \to \dots \to \text{Object.prototype} \to \text{null}$$ 2. **Arrow Function this Invariant:** $$\text{ArrowFunction.this} \equiv \text{EnclosingLexicalScope.this}$$

· 3. Step-by-Step Solved Examples

Example 2.1: Creating a Private Counter via Closures

- **Difficulty / Target Companies:** `Easy | Amazon, Microsoft, TCS`
- **Problem Statement:** Create a function `createCounter()` that encapsulates state, exposing `increment()`, `decrement()`, and `getValue()` without allowing direct modification of the count variable.

Step-by-Step Walkthrough & Logic:

- Step 1: Declare private variable `count = 0` inside outer function.

- Step 2: Return an object containing closure methods that read and mutate `count`.
- Step 3: External code cannot access `count` directly.

Complete Solved Code / Calculation:

```
function createCounter(initialValue = 0) {
    let count = initialValue; // Enclosed private state

    return {
        increment() { return ++count; },
        decrement() { return --count; },
        getValue() { return count; }
    };
}

const counter = createCounter(10);
console.log(counter.increment()); // 11
console.log(counter.getValue());  // 11
// counter.count is undefined (strictly encapsulated!)
```

- **Line-by-Line Breakdown & Complexity Analysis:** The inner methods maintain a persistent reference to the enclosing `count` variable, providing robust data privacy.

· 4. Practice Questions with Answers & Explanations

Practice Question 2.1

Question: What is the output of `(function(){ var a = b = 3; })(); console.log(typeof a, typeof b);` in non-strict mode?

Options:

- A) undefined, number
- B) number, number
- C) undefined, undefined
- D) Throws ReferenceError

Answer: **A) undefined, number** **Detailed Solution & Reasoning:** `var a = b = 3` evaluates right-to-left: `b = 3` creates an implicit global variable, while `var a` is scoped locally to the IIFE.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 2.1 (Google)

- **Interview Question:** _What is the difference between Function.prototype.call, apply, and bind?_

- **Model High-Scoring Answer:** `call()` and `apply()` invoke the function immediately with explicit `this` binding. `call()` takes arguments separated by commas (`fn.call(ctx, arg1, arg2)`), while `apply()` takes arguments as an array (`fn.apply(ctx, [arg1, arg2])`). `bind()` does not execute the function immediately; it returns a new bound function with `this` permanently locked to the provided context.

- **Interviewer Follow-Up & Pro Tip:** ? Mnemonic: 'A for Array (apply), C for Comma (call)'.

CHAPTER 3

Asynchronous JavaScript: Promises & async/await

Key Principle / Exam Takeaway: Master Promise lifecycle (pending, fulfilled, rejected), Promise.all vs allSettled vs race vs any, and async/await error handling.

Core Curriculum Topics Covered in this Chapter:

- Callback Hell and inversion of control
- Promise states: Pending, Fulfilled, Rejected
- Promise chaining, error propagation, and .finally()
- Promise combinators: Promise.all, Promise.allSettled, Promise.race, Promise.any
- async/await syntactic sugar and try/catch error boundaries

· 1. Core Concepts & In-Depth Explanation

Asynchronous JavaScript evolved from nested callbacks ('Callback Hell') to Promises and `async/await`:

- **Promise:** An object representing the eventual completion (or failure) of an asynchronous operation. A Promise is in one of three mutually exclusive states: `pending`, `fulfilled`, or `rejected`. Once settled, its state and value are immutable.

- **async/await:** Built on top of Promises and Generators. Marking a function `async` guarantees it returns a Promise. The `await` keyword pauses execution inside the async function until the Promise settles, resuming execution in the Microtask queue without blocking the main browser thread.

· 2. Key Formulas, Rules & Complexity Classes

1. **Promise Combinator Matrix:**

- **Promise.all:** Rejects immediately if ANY promise rejects (Fail-Fast).
- **Promise.allSettled:** Waits for ALL promises; returns status and value/reason for every item.

- **Promise.race:** Settles as soon as the FIRST promise settles (fulfillment or rejection).

- **Promise.any:** Fulfills as soon as the FIRST promise fulfills; rejects only if ALL reject (AggregateError).

· 3. Step-by-Step Solved Examples

Example 3.1: Robust Concurrent API Fetching with Promise.allSettled

- **Difficulty / Target Companies:** `Easy | Amazon, TCS Digital`
- **Problem Statement:** Fetch user profile and user permissions concurrently, ensuring that failure of one API does not crash the other.

Step-by-Step Walkthrough & Logic:

- Step 1: Execute promises in parallel using `Promise.allSettled()`.
- Step 2: Inspect `status === 'fulfilled'` for each result.

Complete Solved Code / Calculation:

```
async function fetchUserData(userId) {
    const [profileRes, permsRes] = await Promise.allSettled([
        fetch(`/api/users/${userId}`),
        fetch(`/api/users/${userId}/permissions`)
    ]);

    const profile = profileRes.status === 'fulfilled' ? await profileRes.value.json() : null;
    const permissions = permsRes.status === 'fulfilled' ? await permsRes.value.json() : [];

    return { profile, permissions };
}
```

- **Line-by-Line Breakdown & Complexity Analysis:** Promise.allSettled guarantees that partial failures do not prevent successful APIs from rendering.

· 4. Practice Questions with Answers & Explanations

Practice Question 3.1

Question: What happens if one promise in `Promise.all([p1, p2, p3])` rejects?

Options:

- A) It waits for all to finish then reports error
- B) It rejects immediately with that error, ignoring other pending promises
- C) It retries the rejected promise
- D) Returns undefined

Answer: **B) It rejects immediately with that error, ignoring other pending promises** **Detailed Solution & Reasoning:** Promise.all implements fail-fast behavior, immediately rejecting upon the first rejection.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 3.1 (Amazon)

- **Interview Question:** _Can you use await outside an async function?_
- **Model High-Scoring Answer:** Traditionally, `await` was strictly disallowed outside `async` functions. However, modern JavaScript supports **Top-Level Await** in ECMAScript Modules (ESM). In ES modules, `await` can be used at the top level of a module to initialize asynchronous resources before dependent modules execute.
- **Interviewer Follow-Up & Pro Tip:** ? Clarify that top-level await is valid in ES Modules (.mjs or type='module').

CHAPTER 4

Modern ES6+ Features & Functional Programming

Key Principle / Exam Takeaway: Master destructuring, rest/spread operators, Map/Set, WeakMap, Array higher-order functions (map, filter, reduce), and immutability.

Core Curriculum Topics Covered in this Chapter:

- Destructuring assignment: Objects, arrays, defaults, and aliases
- Rest parameters (...args) vs Spread operator (...iterable)
- Array functional methods: map, filter, reduce, some, every, flatMap
- Map and Set vs Object and Array, WeakMap for private data and memory safety
- Pure functions, immutability, and function currying

· 1. Core Concepts & In-Depth Explanation

ECMAScript 6 (ES2015) transformed JavaScript into a modern, expressive language:

- **Destructuring:** Clean extraction of properties into distinct variables: `const { name: userName, age = 18 } = user`.
- **Spread & Rest (`...`):** Rest gathers function arguments into an array; Spread expands arrays or objects into elements.
- **Higher-Order Array Methods:**
- `map`: Transforms each element 1-to-1.
- `filter`: Extracts elements satisfying a boolean predicate.
- `reduce`: Accumulates an array into a single scalar, object, or grouped dictionary.
- **WeakMap / WeakSet:** Keys must be objects and are held weakly. If no other references to a key object exist, it is garbage collected, preventing memory leaks in DOM metadata associations.

· 2. Key Formulas, Rules & Complexity Classes

1. **Array Reduce Accumulator Invariant:** $$\text{arr.reduce}((acc, curr) \implies \text{nextAcc}, \text{initialVal})$$

· 3. Step-by-Step Solved Examples

Example 4.1: Grouping and Aggregating Data with Array.prototype.reduce

- **Difficulty / Target Companies:** `Easy | Amazon, Flipkart`
- **Problem Statement:** Given a list of transactions, calculate total spent per category using reduce.

Step-by-Step Walkthrough & Logic:

- Step 1: Start with empty object `{}` accumulator.
- Step 2: Add transaction amount to category tally.
- Step 3: Return accumulator.

Complete Solved Code / Calculation:

```
const transactions = [
    { category: 'Food', amount: 20 },
    { category: 'Transport', amount: 15 },
    { category: 'Food', amount: 35 },
];

const totalByCategory = transactions.reduce((acc, { category, amount }) => {
    acc[category] = (acc[category] || 0) + amount;
    return acc;
}, {});

console.log(totalByCategory); // { Food: 55, Transport: 15 }
```

- **Line-by-Line Breakdown & Complexity Analysis:** Replaces imperative for-loops with a clean, single-pass functional aggregation.

· 4. Practice Questions with Answers & Explanations

Practice Question 4.1

Question: What is the key difference between Map and a standard JavaScript Object?

Options:

- A) Objects are faster for iteration
- B) Map keys can be of any data type (including objects and functions), while Object keys must be strings or symbols
- C) Objects preserve insertion order, Maps do not
- D) Map cannot be converted to JSON

Answer: **B) Map keys can be of any data type (including objects and functions), while Object keys must be strings or symbols**
Detailed Solution & Reasoning: Map allows arbitrary object references as keys and provides a direct `.size` property.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 4.1 (Microsoft)

- **Interview Question:** _What is a Pure Function and why is immutability important in React/Redux architectures?_
- **Model High-Scoring Answer:** A Pure Function is deterministic (always returns identical output for identical inputs) and has zero side effects (does not mutate external variables, make I/O calls, or modify parameters). Immutability is crucial in modern frameworks like React because state changes can be detected via fast shallow reference equality checks (`prevProps !== nextProps`) in $O(1)$ time, avoiding expensive deep object comparisons.
- **Interviewer Follow-Up & Pro Tip:** ? Mention shallow reference comparison ($O(1)$) in Virtual DOM diffing.

CHAPTER 5

TypeScript Type System & Strict Safety

Key Principle / Exam Takeaway: Master static typing, Interfaces vs Types, Generics, Union & Intersection types, Type Narrowing, and utility types (Partial, Pick, Omit).

Core Curriculum Topics Covered in this Chapter:

- TypeScript compilation: Transpilation to JavaScript and type erasure
- Interfaces vs Type Aliases (Declaration merging, union capabilities)
- Generics: Constrained generics (<T extends { id: string }>) and generic functions
- Type Narrowing: typeof, instanceof, in operator, and Discriminated Unions
- Utility Types: Partial, Required, Readonly, Pick, Omit, Record, ReturnType

· 1. Core Concepts & In-Depth Explanation

TypeScript is a typed superset of JavaScript that compiles to plain JavaScript. All type annotations are stripped during compilation (**Type Erasure**), introducing zero runtime performance overhead.

Key Type System Features:

- **Interfaces vs Types:** Interfaces support **Declaration Merging** (extending third-party library definitions) and represent object contracts. Types support primitives, unions (`A | B`), intersections (`A & B`), and mapped types.
- **Discriminated Unions:** Pattern where each type in a union shares a common literal discriminator property (e.g., `kind: 'circle' | 'square'`), enabling exhaustiveness checks via `switch`.
- **Type Narrowing:** Refining broad types to specific subtypes using type guards (`typeof`, `instanceof`, custom `is` predicates).

· 2. Key Formulas, Rules & Complexity Classes

1. **Discriminated Union Structure:** $$\text{type Shape} = \{ \text{kind: 'circle', radius: number} \} \mid \{ \text{kind: 'square', size: number} \}$$ 2. **Key Utility Types:** $$\text{Pick<T, K>} = \text{Extracts subset } K \text{ of keys from } T$$ $$\text{Omit<T, K>} = \text{Drops keys } K \text{ from } T$$

· 3. Step-by-Step Solved Examples

Example 5.1: Generic API Response Wrapper with Type Constraints

- **Difficulty / Target Companies:** `Easy | Microsoft, Uber`
- **Problem Statement:** Create a generic API response type that handles success and error payloads safely.

Step-by-Step Walkthrough & Logic:

- Step 1: Define generic interface `ApiResponse<T>`.
- Step 2: Use optional generic parameter with default type.

Complete Solved Code / Calculation:

```
interface ApiResponse<T> {
    success: boolean;
    data: T | null;
    error?: string;
    timestamp: number;
}

interface User {
    id: number;
    name: string;
}

// Type-safe consumption
const response: ApiResponse<User> = {
    success: true,
    data: { id: 101, name: "Alice" },
    timestamp: Date.now()
};
```

- **Line-by-Line Breakdown & Complexity Analysis:** Provides compile-time safety ensuring `data` matches the expected type contract.

· 4. Practice Questions with Answers & Explanations

Practice Question 5.1

Question: What is the difference between `unknown` and `any` in TypeScript?

Options:

- A) They are identical
- B) any turns off type checking; unknown requires type narrowing before performing operations
- C) unknown only accepts strings
- D) any cannot be assigned to objects

Answer: **B) any turns off type checking; unknown requires type narrowing before performing operations** **Detailed Solution & Reasoning:** unknown is the type-safe counterpart of any, requiring explicit type guards prior to access.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 5.1 (Microsoft)

- **Interview Question:** _What is Declaration Merging in TypeScript and when should you use an Interface over a Type?_
- **Model High-Scoring Answer:** Declaration Merging occurs when the TypeScript compiler merges two separate interfaces with the identical name into a single definition. It is essential when augmenting existing global libraries or window objects (e.g., adding custom properties to `Window` or `Express.Request`). Use an Interface when building public API libraries or object contracts that require extension. Use a Type for unions, tuples, primitives, or mapped transformations.
- **Interviewer Follow-Up & Pro Tip:** ? Cite extending Express `req.user` as a practical production example.

CHAPTER 6

DOM Manipulation, Browser APIs & Event Propagation

Key Principle / Exam Takeaway: Master the DOM tree, Event Bubbling vs Capturing, Event Delegation, Debounce vs Throttle, and IntersectionObserver.

Core Curriculum Topics Covered in this Chapter:

- DOM tree representation, querySelector, and DocumentFragment for batch mutations
- Event Propagation: Capturing phase, Target phase, and Bubbling phase
- Event Delegation: Single listener on parent element for dynamic children
- Rate limiting: Debouncing (search input) vs Throttling (scroll events)
- Modern Web APIs: IntersectionObserver (lazy loading) and LocalStorage vs SessionStorage vs Cookies

· 1. Core Concepts & In-Depth Explanation

The Document Object Model (DOM) is the tree representation of an HTML document in memory. Manipulating the DOM triggers expensive browser **Reflow** (recalculating geometric layout) and **Repaint** (drawing pixels). To optimize performance, batch DOM updates using `DocumentFragment`.

Event Propagation (3 Phases): 1. **Capturing Phase:** Event trickles down from `window` through ancestors to target. 2. **Target Phase:** Event reaches the target element. 3. **Bubbling Phase:** Event bubbles up from target back through ancestors to `window`.

Event Delegation: Instead of attaching separate event listeners to 1,000 table rows, attach a single listener to the parent container (`<table>`), inspecting `e.target` during the bubbling phase. This saves memory and automatically handles dynamically created elements.

· 2. Key Formulas, Rules & Complexity Classes

1. **Rate Limiting Invariants:**

- **Debounce:** Resets timer on every call. Executes only after a quiet period of X milliseconds (e.g., search autocomplete).
- **Throttle:** Executes at most once per fixed time interval X (e.g., scroll/resize).

· 3. Step-by-Step Solved Examples

Example 6.1: Production Debounce Implementation

- **Difficulty / Target Companies:** `Medium | Amazon, Google, Flipkart`
- **Problem Statement:** Implement a custom debounce function from scratch in vanilla JavaScript.

Step-by-Step Walkthrough & Logic:

- Step 1: Return a closure maintaining a `timerId`.
- Step 2: Clear existing timer on each invocation.

- Step 3: Schedule new timer with `setTimeout` invoking callback with arguments.

Complete Solved Code / Calculation:

```
function debounce(func, delayMs) {
    let timerId = null;
    return function (...args) {
        if (timerId) clearTimeout(timerId);
        timerId = setTimeout(() => {
            func.apply(this, args);
        }, delayMs);
    };
}

// Usage:
const handleSearch = debounce((query) => {
    console.log("Searching for:", query);
}, 300);
```

- **Line-by-Line Breakdown & Complexity Analysis:** Guarantees that rapid keystrokes trigger only a single network API call after the user stops typing.

· 4. Practice Questions with Answers & Explanations

Practice Question 6.1

Question: What is the primary benefit of Event Delegation?

Options:

- A) Faster network requests
- B) Reduced memory overhead and automatic handling of dynamically added child elements
- C) Prevents event bubbling
- D) Eliminates CSS animations

Answer: **B) Reduced memory overhead and automatic handling of dynamically added child elements** **Detailed Solution & Reasoning:** A single listener on the parent leverages bubbling to intercept events from any child without per-element allocations.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 6.1 (Amazon)

- **Interview Question:** _What is the difference between Reflow and Repaint in browser rendering?_
- **Model High-Scoring Answer:** Reflow (Layout) is the process where the browser calculates the positions and geometries of all elements in the document tree. It is computationally expensive. Modifying properties like `width`, `height`, `margin`, or `fontSize` triggers reflow. Repaint occurs when an element's visual appearance changes without altering layout geometry (e.g., `color`, `background-color`, `visibility`). Reflow always triggers a subsequent repaint, but repaint can occur without reflow.
- **Interviewer Follow-Up & Pro Tip:** ? Mention using `transform` and `opacity` because they execute on the GPU compositor thread without triggering reflow.

CHAPTER 7

Modules, Bundlers & Memory Management

Key Principle / Exam Takeaway: Master ES Modules (import/export) vs CommonJS (require), Tree Shaking, Garbage Collection (Mark and Sweep), and memory leaks.

Core Curriculum Topics Covered in this Chapter:

- Module systems: CommonJS (synchronous, runtime) vs ES Modules (static, compile-time)
- Tree Shaking: Dead code elimination via static ESM analysis
- V8 Garbage Collection: Generational Mark and Sweep with Scavenger
- Common JavaScript memory leaks: Forgotten timers, dangling event listeners, detached DOM nodes
- Chrome DevTools Memory Profiler (Heap Snapshots, Allocation Timelines)

· 1. Core Concepts & In-Depth Explanation

JavaScript modularity transitioned from CommonJS (`require()`, Node.js legacy) to official **ES Modules (ESM)** (`import / export`):

- **CommonJS:** Dynamic and evaluated at runtime. Modules are loaded synchronously.
- **ES Modules:** Static structure determined at compile time. Enables bundlers (Webpack, Vite, Rollup) to perform **Tree Shaking**: automatically identifying and stripping unused exported functions from production bundles.

Memory Leaks: V8 uses generational Mark-and-Sweep garbage collection. Memory leaks occur when active references prevent unused memory from being reclaimed: 1. Forgotten `setInterval()` references retaining large scope variables. 2. Event listeners attached to DOM elements that are removed from the page without `.removeEventListener()`. 3. Detached DOM nodes retained in global JavaScript arrays.

· 2. Key Formulas, Rules & Complexity Classes

1. **Tree Shaking Feasibility:** $$\text{Static ESM (import { fn } from 'mod')} \implies \text{Tree Shakeable}$$ $$\text{Dynamic CommonJS (const mod = require('mod'))} \implies \text{Cannot be Tree Shaken}$$

· 3. Step-by-Step Solved Examples

Example 7.1: Detecting and Fixing Detached DOM Node Leaks

- **Difficulty / Target Companies:** `Medium | Google, Uber`
- **Problem Statement:** Identify the memory leak in code appending DOM elements to a global array, and provide the fix.

Step-by-Step Walkthrough & Logic:

- Step 1: Storing removed DOM elements in global variables keeps the entire subtree alive in Heap memory.
- Step 2: Clear references or use WeakRef / WeakSet.

Complete Solved Code / Calculation:

```
// LEAKY CODE:
const leakedElements = [];
function createAndRemove() {
    const btn = document.createElement("button");
    document.body.appendChild(btn);
    leakedElements.push(btn); // Retains reference!
    document.body.removeChild(btn); // Detached from DOM, but retained in Heap!
}

// FIXED CODE:
function cleanCreateAndRemove() {
    const btn = document.createElement("button");
    document.body.appendChild(btn);
    // Perform operations...
    document.body.removeChild(btn);
    // Do NOT retain reference in global array; V8 reclaims memory automatically
}
```

- **Line-by-Line Breakdown & Complexity Analysis:** Removing an element from the DOM does not free memory if references still exist in JavaScript arrays.

· 4. Practice Questions with Answers & Explanations

Practice Question 7.1

Question: Why can ES Modules be tree-shaken by bundlers while CommonJS cannot?

Options:

- A) ESM files are smaller
- B) ESM imports/exports are static and known at build time, allowing dead code elimination
- C) CommonJS is deprecated in browsers
- D) ESM runs on multiple threads

Answer: **B) ESM imports/exports are static and known at build time, allowing dead code elimination** **Detailed Solution & Reasoning:** Static analysis allows compilers to determine with certainty which exports are never referenced.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 7.1 (Google)

- **Interview Question:** _How do you identify a memory leak in a single-page web application using Chrome DevTools?_
- **Model High-Scoring Answer:** Open Chrome DevTools Memory tab. 1) Take a baseline Heap Snapshot. 2) Perform the suspected leaky user interaction (e.g., opening and closing a dialog 10 times). 3) Take a second Heap Snapshot. 4) Use the 'Comparison' view between Snapshot 2 and Snapshot 1. Filter by 'Objects Allocated between Snapshot 1 and 2' or search for 'Detached HTML elements'. Inspect the Retainer tree to identify the root variable preventing garbage collection.
- **Interviewer Follow-Up & Pro Tip:** ? Cite Heap Snapshots and the Retainer tree.

CHAPTER 8

Solved Frontend Placement Coding Drills & System Design

Key Principle / Exam Takeaway: Master custom implementations of deep clone, Promise.all, memoize, flatten array, and frontend architecture design.

Core Curriculum Topics Covered in this Chapter:

- Deep Clone handling circular references and non-serializable types (structuredClone vs custom)
- Custom Promise.all polyfill implementation
- Function memoization cache decorator
- Flatten deeply nested arrays (Array.prototype.flat polyfill)
- Frontend System Design: Autocomplete Search component with debounce, caching, and race condition prevention

· 1. Core Concepts & In-Depth Explanation

This capstone chapter covers machine coding rounds and technical interviews for frontend and full-stack software engineering roles.

Key polyfills and utility algorithms:

- **Deep Clone:** Recursively clones objects while tracking visited objects via `WeakMap` to avoid infinite recursion on circular graphs.
- **Promise.all Polyfill:** Maintains an output array and a counter of completed promises, resolving when `completedCount === promises.length`.
- **Race Condition Prevention in Autocomplete:** If request A is sent before request B, but request A takes longer and responds after request B, request A's stale data must not overwrite request B's newer data. Mitigated using request sequence tokens or `AbortController`.

· 2. Key Formulas, Rules & Complexity Classes

1. **Promise.all Polyfill Resolution:** $$\text{Resolve Output} \iff \text{completedCount} === \text{promises.length}$$

· 3. Step-by-Step Solved Examples

Example 8.1: Implement Custom Promise.all Polyfill

- **Difficulty / Target Companies:** `Hard | Google, Amazon, Uber`
- **Problem Statement:** Implement a function `promiseAll(promises)` that behaves identically to `Promise.all`.

Step-by-Step Walkthrough & Logic:

- Step 1: Return a new Promise.
- Step 2: If input is empty, resolve with `[]` immediately.
- Step 3: Track `completedCount = 0` and `results = []`.
- Step 4: Iterate promises with index. Wrap with `Promise.resolve(p)` to handle raw values.

- Step 5: On fulfill, store `results[i] = val` and increment counter. If counter equals length, resolve `results`.
- Step 6: On any rejection, reject outer promise immediately.

Complete Solved Code / Calculation:

```
function promiseAll(promises) {
    return new Promise((resolve, reject) => {
        if (!Array.isArray(promises)) {
            return reject(new TypeError("Input must be an array"));
        }
        if (promises.length === 0) {
            return resolve([]);
        }

        const results = new Array(promises.length);
        let completed = 0;

        promises.forEach((p, index) => {
            Promise.resolve(p).then(
                (val) => {
                    results[index] = val;
                    completed += 1;
                    if (completed === promises.length) {
                        resolve(results);
                    }
                },
                (err) => {
                    reject(err); // Rejects immediately on first error
                }
            );
        });
    });
}
```

- **Line-by-Line Breakdown & Complexity Analysis:** Properly preserves original index order and implements fail-fast rejection.

· 4. Practice Questions with Answers & Explanations

Practice Question 8.1

Question: Why is `JSON.parse(JSON.stringify(obj))` insufficient for deep cloning arbitrary objects?

Options:

- A) It is slow and drops undefined, functions, Symbols, and throws errors on circular references
- B) It cannot clone numbers
- C) It only works on arrays
- D) It mutates the original object

Answer: **A) It is slow and drops undefined, functions, Symbols, and throws errors on circular references** **Detailed Solution & Reasoning:** JSON serialization fails on circular references and cannot preserve dates, functions, or undefined.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 8.1 (Google)

- **Interview Question:** _How do you handle race conditions in search autocomplete when an earlier slow network request finishes after a later request?_
- **Model High-Scoring Answer:** Two primary techniques: 1) **AbortController:** Before firing a new search request, abort the previous pending request using `controller.abort()`. 2) **Request Sequence ID / Timestamp:** Maintain an integer counter `currentRequestId`. Increment it with each keystroke. Pass the ID into the async closure. When the network response returns, check if its ID matches `currentRequestId`; if it is stale, ignore the response.

- **Interviewer Follow-Up & Pro Tip:** ? Recommend AbortController as the modern standard browser API.

· Campus Placement Certification & Final Exam Policy

To earn Placement-Ready Master status, students must complete all chapter practice questions, review all solved examples, and score at least 80% on the comprehensive subject assessment.